Academic Year: 2026-2027
Semester: III
Class / Branch: SE IT
Subject: ADSAL
Name of Instructor: Ms. Sneha Dalvi

Name of Student:Nishant Suryarao
Student ID:26204001
Date of Performance:22/09/2026
Date of Submission:22/09/2026

Experiment No.3

Aim: To Implement Insert, Delete, Search and Display operations on Binary Search Tree and analyze space and time complexity.

Code:

```
#include <stdio.h>

#include <stdlib.h>


// Structure for BST Node

struct Node

{

   int data;

   struct Node *left, *right;

};


// Create a new node struct

Node* createNode(int value)

{
```

```
   struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
newNode->data = value;     newNode->left = NULL;     newNode->right = NULL;

   return newNode;
}

// Insert a node
struct Node* insert(struct Node* root, int value)
{
   if (root == NULL)
return createNode(value);

   if (value < root->data)        root->left =
insert(root->left, value);     else if (value >
root->data)        root->right = insert(root-
>right, value);

   return root;
}

// Search a node
struct Node* search(struct Node* root, int value)
```

```
{
   if (root == NULL || root->data == value)
return root;

   if (value < root->data)       return
search(root->left, value);

   return search(root->right, value);
}

// Find minimum node
struct Node* minValueNode(struct Node* node)
{
   struct Node* current = node;

   while (current != NULL && current->left != NULL)
current = current->left;

   return current;
}

// Delete a node
```

```
struct Node* deleteNode(struct Node* root, int value)
{
   if (root == NULL)
return root;

   if (value < root->data)
   {
      root->left = deleteNode(root->left, value);
   }
   else if (value > root->data)
   {
      root->right = deleteNode(root->right, value);
   }
else
   {
      // Node with no child or one child
if (root->left == NULL)
      {
         struct Node* temp = root-
>right;          free(root);          return
temp;
      }
```

```
        else if (root->right == NULL)
        {
            struct Node* temp = root-
>left;          free(root);          return
temp;
        }

        // Node with two children
        struct Node* temp = minValueNode(root->right);

        root->data = temp->data;

        root->right = deleteNode(root->right, temp->data);
    }

    return root;
}

// Display BST using Inorder Traversal void
inorder(struct Node* root)
{
    if (root != NULL)
```

```
  {
     inorder(root->left);
     printf("%d ", root->data);       inorder(root-
>right);
  }
}

// Main
Function int
main() {
  struct Node* root = NULL;
int choice, value;

  do
  {
     printf("\n\n--- Binary Search Tree ---");
printf("\n1. Insert");       printf("\n2.
Delete");       printf("\n3. Search");
printf("\n4. Display (Inorder)");
printf("\n5. Exit");       printf("\nEnter your
choice: ");       scanf("%d", &choice);
```

```
    switch (choice)
    {
case 1:
        printf("Enter value: ");
scanf("%d", &value);

        root = insert(root, value);
printf("Element inserted successfully.\n");
break;

      case 2:
        printf("Enter value to delete: ");
scanf("%d", &value);

        if (search(root, value))
        {
          root = deleteNode(root, value);
printf("Element deleted successfully.\n");
        }
else
        {
          printf("Element not found.\n");
```

```
            }
break;
case 3:
            printf("Enter value to search: ");
scanf("%d", &value);

            if (search(root, value))
printf("Element Found.\n");              else
               printf("Element Not Found.\n");
break;

         case 4:
            printf("BST (Inorder): ");
inorder(root);
printf("\n");              break;

         case 5:
            printf("Program Exited.\n");
break;

         default:
```

```
            printf("Invalid Choice!\n");
        }

    } while (choice != 5);

    return 0;
}
```

Output Screen Shots:

```
apsit@apsit-HP-Pro-Tower-280-G9-E-PCI-Desktop-PC:~/Desktop/Nishant$ gcc -o Exp3 Exp3.c
apsit@apsit-HP-Pro-Tower-280-G9-E-PCI-Desktop-PC:~/Desktop/Nishant$ ./Exp3


--- Binary Search Tree ---
1. Insert
2. Delete
3. Search
4. Display (Inorder)
5. Exit
Enter your choice: 1
Enter value: 10
Element inserted successfully.


--- Binary Search Tree ---
1. Insert
2. Delete
3. Search
4. Display (Inorder)
5. Exit
Enter your choice: 1
Enter value: 20
Element inserted successfully.


--- Binary Search Tree ---
1. Insert
2. Delete
3. Search
4. Display (Inorder)
```

```
4. Display (Inorder)
5. Exit
Enter your choice: 1
Enter value: 30
Element inserted successfully.


--- Binary Search Tree ---
1. Insert
2. Delete
3. Search
4. Display (Inorder)
5. Exit
Enter your choice: 4
BST (Inorder): 10 20 30


--- Binary Search Tree ---
1. Insert
2. Delete
3. Search
4. Display (Inorder)
5. Exit
Enter your choice: 3
Enter value to search: 20
Element Found.


--- Binary Search Tree ---
1. Insert
2. Delete
3. Search
4. Display (Inorder)
5. Exit
Enter your choice: 5
Program Exited.
apsit@apsit-HP-Pro-Tower-280-G9-E-PCI-Desktop-PC:~/Desktop/Nishant$
```

Conclusion: We successfully implemented insertion, deletion, searching, and traversal operations on a Binary Search Tree. The experiment helped us understand how BST organizes data and performs operations efficiently based on the tree height.

Experiment 3: Insert, Delete, Search and Display in BST

Definition: A Binary Search Tree supports efficient insertion, deletion, searching, and traversal of elements while maintaining the BST property.

Algorithm:

1. Create an empty BST.
2. Display a menu for Insert, Delete, Search, Display and Exit.
3. For Insert, compare the value with the root and insert it at the correct position.
4. For Delete, find the node and delete it according to its number of children.
5. For Search, compare the key with the current node and move left or right.
6. For Display, perform Inorder Traversal.
7. Repeat until the user selects Exit.
8. Stop.

Deletion cases to remember:

- No child: Delete the node directly.
- One child: Replace the node with its child.
- Two children: Replace the node with its inorder successor (minimum value in the right subtree).

Conclusion: Thus, insertion, deletion, searching, and display operations were successfully implemented on a Binary Search Tree.

Complexity: Average search/insertion/deletion $O(\log n)$; worst case $O(n)$.